



The Set Orienteering Problem (SOP)

Shiraz University
Department of Computer Science
Parsa Bordbar 40435340
Dr. K.Zearati
April 11, 2026

The Set Orienteering Problem (SOP) is a challenging combinatorial optimization and routing problem, acting as a complex generalization of the classic Orienteering Problem. In the SOP, the vertices of a given graph are partitioned into distinct, disjoint sets (or clusters). Each set carries a specific profit. The primary objective is to construct a path—starting from a designated origin and concluding at a specific destination—that maximizes the total collected profit. This routing is strictly bounded by a predefined cost or time budget, denoted as T_{max} . Crucially, the profit of a given cluster can only be collected once, and the route may visit at most one vertex from any given set.

Because SOP is strongly NP-hard, exact mathematical solvers rapidly become computationally intractable as instance sizes grow. To overcome this, this report presents a highly optimized metaheuristic solver based on *Simulated Annealing* (SA). Inspired by the metallurgical process of heating and controlled cooling of materials to minimize thermodynamic free energy, the SA algorithm intelligently navigates the SOP solution space. It strategically escapes local optima by occasionally accepting worse (non-improving) solutions based on a temperature-dependent probability, driven by the Metropolis criterion $e^{-\Delta/T}$. As the search progresses and the temperature slowly decreases, the algorithm smoothly transitions from broad exploration of the search space to aggressive local exploitation.

1. Representation & Data Structures

To achieve maximum execution speed with zero reliance on external libraries, the architecture utilizes contiguous memory blocks, sequential operations, and cache-friendly data structures.

The Problem Struct

The Problem struct holds the core components of the parsed instance. To bypass the memory fragmentation and pointer-chasing overhead of nested vectors (e.g., `Vec<Vec<f64>>`), the distance matrix is flattened into a 1D array (`dist`). Finding the distance between node i and node j is strictly an $O(1)$ operation using the formula $index = i \times num_nodes + j$. The struct also stores cluster memberships, individual cluster profits, and the strict upper limit for travel time T_{max} .

The Solution Struct

The Solution struct represents a routing state. Rather than dynamically recomputing sequences from scratch, it maintains two sequential vectors: `tour_nodes` and `tour_clusters`. It also caches the *total_cost* and *total_profit*. During search iterations, we do not perform full recalculations of the $O(N)$ route. Instead, we compute localized $O(1)$ deltas for proposed moves and apply the differences to the cached totals.

2. Algorithmic Phases

Phase 1.1: Constructive Heuristic

A robust initial solution provides the metaheuristic with a strong starting boundary. The initial construction (`solver/phase1.rs`) employs a greedy strategy:

1. Clusters are sorted in descending order of their available profit.
2. For each cluster, the algorithm selects the most promising node that guarantees the minimum addition to the total travel cost.
3. A cluster is only inserted if the resulting *total_cost* remains strictly $\leq T_{max}$

Phase 1.2: Local Search & Tour Improvement

Once a base sequence is constructed, it is subjected to immediate tour improvement techniques before Simulated Annealing begins:

1. **Vertex Selection (Dynamic Programming):** Given a fixed sequence of clusters, a DP algorithm calculates the optimal exact sequence of nodes that minimizes the total travel cost.
2. **2-Opt Routing:** A 2-opt edge-exchange mechanism untangles crossing edges. Reversing a segment in `tour_nodes` and its corresponding segment in `tour_clusters` significantly optimizes spatial routing.

Phase 2: Simulated Annealing

The core optimization phase relies on a heavily tuned Simulated Annealing (SA) algorithm designed to mirror the findings of the recent literature on SOP.

◆ Neighborhoods:

The search explores an expanded geometry uniformly selecting from multiple moves:

◆ **Insertion**: Moving a node/cluster to a different point in the sequence.

◆ **Drop**: Removing a cluster to free up capacity.

◆ **Replace**: Swapping a cluster inside the route with an unvisited cluster outside.

◆ **Swap (15%)**: Exchanging two positions i and j inside the tour.

◆ **Inversion / 2-Opt (20%)**: Reversing a sub-segment $[p1..=p2]$.

◆ **Node Change (15%)**: Swapping an active node for a different node belonging to the same cluster.

All evaluations are done using $O(1)$ delta math to evaluate millions of states per second.

◆ **Evaluation Function & Penalties:**

The objective dynamically handles constraints. Instead of discarding infeasible moves, they are permitted but heavily penalized. The evaluation applies a penalty factor of 100.0:

$$Objective = Profit - 100.0 \times \max(0, Cost - T_{max})$$

Cooling Schedule & Parameters. Following tuned best-practices, the hyperparameters are fixed to:

- $T_0 = 16.0$ (Initial Temperature)
- $\alpha = 0.93$ (Geometric Cooling rate: $T_{k+1} = \alpha \cdot T_k$)
- Epoch Length: 80,000 iterations per temperature level.
- Acceptance uses standard Metropolis criteria: if $\Delta \geq 0$, accept. If $\Delta < 0$, accept with probability $P = \exp(\Delta/T)$.

○ Initialization, Intensification & Diversification:

High initial temperatures combined with large epochs (80,000) heavily diversify the search early on. Because localized $O(1)$ delta calculations accumulate floating-point drift over millions of steps, the system implements a cyclic $O(N)$ `recompute()` every 100,000 iterations to guarantee objective purity.

3. Code Architecture: Modules and Functions

The codebase (located within `sop_simulated_annealing/src/`) is designed with a strict emphasis on modularity, high performance, and separation of concerns. To achieve maximum execution speed and absolute control over memory management, the project relies purely on Rust's standard

library, completely avoiding external third-party crates (dependencies). Every component is optimized for sequential execution.

Below is a detailed breakdown of the project hierarchy and module responsibilities:

3.1. Entry Points and Orchestration

◆ **main.rs & lib.rs:** The central nervous system of the application. `lib.rs` formally structures the internal modules, while `main.rs` acts as the primary execution binary. It parses basic command-line arguments and dictates the flow of execution—either running a deep analysis on a single `.sop` instance or orchestrating a batch run across the entire benchmark suite.

◆ **configs.rs:** Centralizes all hardcoded hyperparameters and system constants. Parameters such as the Simulated Annealing initial temperature (T_0), cooling rate (α), inner loop epoch length, and penalty multipliers are defined here. Separating these from the algorithmic logic ensures clean code and rapid parameter tuning.

◆ **benchmark (Directory / Logic):** Automates the testing pipeline. It systematically iterates over all instances in `data/instances/`, triggers the solver, and aggregates metrics (Best Profit, Associated Cost, and Average Execution Time), outputting the final metrics to a CSV file.

3.2. Data Management and I/O (`io/` and `data/`)

◆ **data/ Directory:** Houses the physical data assets. `data/instances/` contains the raw `.sop` problem sets and the `BKS.xlsx` file, while `data/output/` stores the generated solution routes (e.g., `instance_2_output.txt`) and `benchmark_results.csv`.

○ **io/parser.rs:** Responsible for safely parsing the `.sop` text files. A critical optimization occurs here: the parser immediately translates 1-indexed file data into 0-indexed internal arrays, allowing the solver to use lightning-fast, direct memory array accesses without doing offset math during the SA loops. Furthermore, it precomputes the Euclidean distances using a ceiling function (`CEIL_2D`). This guarantees deterministic, integer-based cost evaluations, entirely eliminating floating-point rounding drifts.

○ **io/output.rs:** Handles the serialization of the final Solution structs. It safely writes the exact node sequence and resulting profit/cost metrics into formatted text and CSV files for post-analysis.

3.3. Core Domain and Validation

◆ **models.rs:** Defines the fundamental data structures: Problem and Solution. To prioritize speed, these models avoid deeply nested pointers or complex object-oriented patterns. Instead, they utilize flat arrays and vectors. Problem stores the precomputed distance matrix and cluster mappings, enabling $O(1)$ lookups. Solution tracks the current permutation of nodes, the active cost, and the accumulated profit.

◆ **feasibility.rs:** The unyielding validator of the system. In metaheuristics, structural integrity is paramount. This module provides strict boolean checks to ensure a proposed route is mathematically legal. It verifies that the route begins and ends exactly at the depot, confirms that node-to-cluster mappings are legitimate (no cluster is visited more than once), and rigorously enforces the T_{max} cost constraint.

3.4. The Algorithmic Engine (solver/)

This directory contains the core intelligence of the application, broken into sequential phases:

◆ **solver/init.rs (Random Feasible Initialization):** Generates a purely random baseline solution. It iteratively selects valid, unvisited nodes until the T_{max} constraint is saturated. While not inherently optimal, it ensures the solver always has a mathematically sound fallback starting state.

◆ **solver/phase1.rs (Constructive Greedy Heuristic):** Instead of starting the SA from a blind random state, this module intelligently constructs a high-quality initial route. It uses a greedy approach, evaluating the highest “profit-

to-distance” ratio for the next available move. This gives the subsequent SA phase a massive head start.

◆ **solver/simulated_annealing.rs:** The absolute heartbeat of the engine. This module houses the outer thermodynamic cooling loop and the inner Metropolis evaluation loop. It is engineered for microscopic latency:

- It implements the Neighborhood Generators (Insertion, Swap, and Inversion). Crucially, it executes $O(1)$ delta evaluations. Instead of recalculating the cost of a full route (which would be $O(N)$), it only calculates the localized difference (the delta) of the swapped or inserted edges.

- It manages the temperature decay schedule and the probabilistic acceptance of worse solutions ($e^{-\Delta/T}$).

- It includes a periodic full-recalculation safeguard every 100,000 iterations to correct any microscopic floating-point errors accumulating from the delta evaluations.

3.5. Utilities

◆ **rng.rs:** A highly optimized, custom Pseudo-Random Number Generator (PRNG). To strictly honor the “no external libraries” rule (avoiding the popular rand crate), this module implements a deterministic algorithm (such as a Linear Congruential Generator or Xorshift). It provides the rapid, uniformly distributed random numbers required for neighborhood selection and Metropolis probability checks. Because it is deterministic, it ensures that every benchmark run is 100% reproducible when given the same seed.

4. Custom PRNG Implementation

Because external dependencies (like the rand crate) are forbidden for this project’s constraints, rng.rs contains a custom, lightweight Pseudo-Random Number Generator. Using a linear congruential generator (LCG) or Xorshift structure, this inline PRNG is blazingly fast. It serves integer distributions for choosing neighbor indices and uniform $[0,1)$ float distributions necessary for the Metropolis acceptance probability $P = \exp(-\Delta/T)$.

5. Performance Optimizations & Execution Speed

Execution speed dictates the extent of search depth. To achieve execution times of ~130ms while evaluating massive epoch lengths:

◆ **Zero Dynamic Allocation:** Inside the main annealing loop, no new Vecs are allocated. Route changes are performed via in-place mutation.

◆ **O(1) Edge Reconnection:** Determining the cost change of a SWAP or INVERSION requires analyzing only the 2 to 4 edges being broken and formed, rather than looping over the N-length arrays.

◆ **Memory Footprint:** Sequential vector alignment ensures that memory blocks are prefetched into the CPU cache (L1/L2), drastically reducing cycle stalls.

6. Benchmark Results

The SA framework successfully tackles complex instances, routinely finishing evaluations in just over a tenth of a second while achieving strong BKS approximations. (Results are base on 3 runs for each problem set)

Instance	SA Best Profit	BKS Profit	SA Best Cost	Avg Time (ms)
instance_2.sop	4413	4550	5271	124
instance_6.sop	9115	9338	10535	131
instance_8.sop	9870	10010	12754	134
instance_11.sop	134	138	7795	139
instance_14.sop	8275	9077	9953	167
instance_16.sop	9769	9990	12559	134
instance_19.sop	147	152	39765	142
instance_27.sop	140	148	915	144
instance_31.sop	214	221	1535	148
instance_33.sop	86	86	25269	136
instance_40.sop	11347	11368	61667	131
instance_41.sop	103	105	397	142
instance_45.sop	221	226	777	150
instance_47.sop	246	258	949	152
instance_50.sop	5681	6423	10616	129
instance_53.sop	209	238	21033	144

instance_58.sop	5203	5630	428	128
instance_64.sop	13213	13971	1033	138
instance_66.sop	6336	6600	8962	125
instance_67.sop	168	187	12774	143
instance_71.sop	287	296	21110	146
instance_77.sop	280	295	14928	145
instance_82.sop	10381	10971	2507	126
instance_86.sop	17546	18681	4856	136
instance_88.sop	19271	20140	5955	139
instance_91.sop	274	290	5747	144
instance_95.sop	408	415	8654	145
instance_99.sop	351	385	29390	145
instance_104.sop	21118	22127	51570	138
instance_107.sop	308	327	12420	139
instance_109.sop	380	402	16585	141
instance_113.sop	267	278	7922	134
instance_118.sop	22913	23769	15581	136
instance_121.sop	245	264	922	145
instance_125.sop	454	499	1787	151
instance_128.sop	26748	28594	2291	141
instance_131.sop	416	447	9385	144
instance_132.sop	20597	22812	9601	133
instance_133.sop	500	541	12631	149
instance_137.sop	303	315	8672	139
instance_141.sop	617	637	20471	173
instance_142.sop	25552	32335	20737	136
instance_144.sop	32824	32991	25250	138

Appendix: Special Rust Syntax & Concepts

To enforce our strict speed and memory parameters, several Rust-specific paradigms are heavily utilized:

◆ **Ownership and Borrowing:** By leveraging mutable references (&mut Solution), the codebase guarantees memory safety without the runtime overhead of a Garbage Collector.

◆ **Slices:** &[T] slices are used for reading segments of routes during 2-Opt inversions, avoiding costly cloning operations.

◆ **Inline Functions:** Mathematical calculations and custom PRNG state updates are tagged with #[inline(always)], instructing the compiler to inject the assembly directly into the call site to eliminate stack-frame overhead.

◆ **Option<T> & Result Types:** Used aggressively during initial node selections to safely prevent null-pointer exceptions without sacrificing runtime speed.

References

1. Primary Research Paper

- ◆ **Lin, S.-W., Guo, S., & Wu, W.-J. (2021).** "Applying the Simulated Annealing Algorithm to the Set Orienteering Problem with Mandatory Visits". **Mathematics**, 9(7), 731. MDPI. [Source: SA.pdf, Page 1]

2. Course Materials

- ◆ Lecture 4: Local Search & Metaheuristics. Course Slides.
- ◆ **Key Concept:** Basic local search principles and neighborhood structures.
- ◆ Lecture 6: Simulated Annealing. Course Slides. [Source: 6- Simulated Annealing.pdf]
- ◆ **Key Concept:** Metropolis Criterion, cooling schedules, and the physical analogy of annealing in metallurgy.
- ◆ Lecture 5: Random Search. Course Slides.
- ◆ **Key Concept:** Stochastic exploration of the search space.

3. Rust Programming Language Documentation

- ◆ Klabnik, S., & Nichols, J. "*The Rust Programming Language*" (The Book). Official Documentation. <https://doc.rust-lang.org/book/>
- ◆ Chapter 4: Understanding Ownership. (Crucial for memory safety without a garbage collector).
- ◆ Chapter 5: Using Structs to Structure Related Data. (Used for the Problem and Solution objects).
- ◆ Chapter 6: Enums and Pattern Matching. (Specific implementation of `Option<T>` for move selection logic).
- ◆ Chapter 13: Functional Language Features: Iterators and Closures. (Used for efficient sorting and data transformation in `phase1.rs`).
- ◆ Rust Standard Library Documentation:
- ◆ `std::vec::Vec`: Performance characteristics of `retain`, `push`, and `remove`.
- ◆ `std::slice`: In-place mutation methods such as `swap` and `reverse` (used in 2-Opt/Inversion moves).